

Reviewer Pack — Minimal Index of Lie Algebra Representations and DPLL Proof ...

Entry 4ece9dd54998 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 15:16:26 UTC

Minimal Index of Lie Algebra Representations and DPLL Proof Width Inequality

Entry ID: 4ece9dd54998 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 15:16:26 UTC

1. Conjecture

Field A (mathematical branch): Lie Theory (Lie Algebras and Representation Theory) **Field B** (complexity object): Complexity Theory (DPLL Proof Complexity)

Statement:

For every Boolean formula φ with n variables, the minimal index of a simple finite-dimensional representation V of the Lie algebra of permutations associated with φ is linearly correlated with its DPLL proof width $w_{\text{DPLL}}(\varphi)$, such that $\min_index(V) = \Omega(w_{\text{DPLL}}(\varphi))$ and $O(\min_index(V)) = \Theta(w_{\text{DPLL}}(\varphi))$.

Rationale (proposer's reasoning):

The minimal index of a representation can be computed using character theory, which provides an invariant that might capture the complexity of DPLL proof construction. Lie algebras associated with Boolean formulas have been studied in representation theory, offering a novel perspective on complexity measures.

Taxonomy category: LieAlgebraRepresentationTheory × ComplexityTheory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 229ae2f9813cdb16

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for at least 95% of randomly generated Boolean formulas φ with n variables, the ratio of the minimal index of a simple finite-dimensional representation V of the permutation Lie algebra to the DPLL proof width $w_{\text{DPLL}}(\varphi)$ meets the conditions: $\min_index(V) / w_{\text{DPLL}}(\varphi) \geq 2.0$ AND $\min_index(V) / w_{\text{DPLL}}(\varphi) \leq 1.5$, and falsified if this ratio is outside these bounds for any seed.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal index of Lie algebra representations" AND "DPLL proof width inequality" - "Lie theory" AND "complexity theory" AND DPLL - "permutation associated with Boolean formula" AND representation AND DPLL proof width

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_formula(n):
        if n == 1:
            return random.choice(['True', 'False'])
        else:
            op = random.choice(['and', 'or'])
            left = generate_boolean_formula(n // 2)
            right = generate_boolean_formula(n - n // 2)
            return f"({left} {op} {right})"

    def dpll_proof_width(phi):
        if phi == "True":
            return 1
        elif phi == "False":
            return None
        else:
            left, op, right = phi.split()
            if op == 'and':
                left_width = dpll_proof_width(left)
                right_width = dpll_proof_width(right)
                if left_width is not None and right_width is not None:
                    return max(left_width, right_width) + 1
                else:
                    return None
            elif op == 'or':
```

```

        left_width = dpll_proof_width(left)
        right_width = dpll_proof_width(right)
        if left_width is not None or right_width is not None:
            return max(left_width, right_width) + 1
        else:
            return None

def minimal_index(phi):
    # Placeholder for the actual computation of the minimal index
    # This is a dummy function and should be replaced with the actual implementation
    return random.randint(1, 10)

n = random.choice([5, 10, 15, 20, 30, 40])
phi = generate_boolean_formula(n)
w_DPLL = dpll_proof_width(phi)
min_index_V = minimal_index(phi)

if w_DPLL is None:
    return {
        "metric_name": "min_index(V) / w_DPLL( $\phi$ )",
        "metric_value": None,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "DPLL proof width is None"
    }

ratio = min_index_V / w_DPLL
return {
    "metric_name": "min_index(V) / w_DPLL( $\phi$ )",
    "metric_value": ratio,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": 2.0 <= ratio <= 1.5,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [random.randint(1, 1000000) for _ in range(30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["metric_value"] is not None for result in results):
        mean_ratio = sum(result["metric_value"] for result in results) / len(results)
        support_fraction = sum(1 for result in results if 2.0 <= result["metric_value"] <= 1.5) / len(results)
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=NA support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if result["metric_value"] is None)
        print(f"RESULT: FALSIFIED counterexample='DPLL proof width is None' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8e9cce72.py", line 88, in <module>
    trial_result = run_trial(seed)
                   ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8e9cce72.py", line 59, in run_trial
    w_DPLL = dpll_proof_width(phi)
             ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8e9cce72.py", line 36, in dpll_proof_wi
    left, op, right = phi.split()
    ~~~~~
```

ValueError: too many values to unpack (expected 3)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test ensuring that it completes without crashing and produces results for analysis.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14026
2	propose	ollama_remote	glm4:latest	0	0	12325
3	preregistration	ollama_remote	glm4:latest	0	0	10250
4	novelty	ollama_remote	glm4:latest	0	0	10407
5	novelty	ollama_remote	glm4:latest	0	0	10356
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18666
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8920
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10368
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9229
10	judge	ollama_remote	glm4:latest	0	0	11377

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 115923 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/4ece9dd54998.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4ece9dd54998.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/4ece9dd54998.tar.gz` (if generated)